BSCS/BSSE/BSDS/BSAI FINAL YEAR PROJECT PROPOSAL

Privacy-Preserving Healthcare GenAI

Term of Registration: Spring 2026

Particulars of the students:

Sr. #	Registration# eg.L1F22BSCS0101	Name in Full Use Block Letters	Contact #	CGPA
1	L1F23BSCS0192	Faisal Khalid	0317-4357289	2.34
2	L1F23BSCS0199	Syed Shah Wali	0334-4690128	2.83
3	L1F23BSCS0204	M. Hammad	0324-4383826	2.41
4	L1F23BSCS0669	Wajeeh ur Rehman	0317-3320965	2.25

Project Type:

Software Development **Artificial Intelligence (AI)**

Faculty of Information Technology & Computer Science

University of Central Punjab

Privacy-Preserving Healthcare GenAI

Write down the brief project topic in four to five words only. However, the topic should not be ambiguous or very general.

Project Advisor

Muhammad Zulkifl Hasan

Acceptance of Project Idea

Students will be required to defend their idea before the scrutiny committee (SC) which has the authority to accept or reject the project idea. The decision taken by the SC will be final and cannot be challenged.

Similarly, in phase wise evaluations, the marks awarded by the evaluators will be considered final. No excuses on their skill, relevancy, competency or biasedness will be acceptable as an absolve.

Supervisory Meetings

A group is required to hold at least two meetings per month and maintain meeting minutes. Missing two consecutive meetings without any notice may lead to withdrawal of the project.

Advisor's Consent

I Prof./Dr./Mr./Ms. _____ am willing to guide these students in all phases of above-mentioned project as advisor. I have carefully seen the Title and description of the project and believe that it is of an appropriate difficulty level for the number of students named above.

Note:
Advisor can't be changed and the duration for completion of the Project is 2 regular semesters (approx.) from the date of Registration of the Project.

- Signatures and Date

Advisor

Student 1:
Name: Faisal Khalid
Registration#
L1F23BSCS0192

_____
Signature

Student 2:
Name: Shah Wali
Registration#
L1F23BSCS0199

_____
Signature

Student 3:
Name: Wajeeh ur Rehman
Registration#
L1F23BSCS0669

_____
Signature

Student 4:
Name: M. Hammad
Registration#
L1F23BSCS0204

_____
Signature

Table of Contents

1. Abstract

Conventional machine learning requires centralizing all user data on a single server a model that fundamentally conflicts with privacy, sovereignty, and security requirements in sensitive domains like healthcare, finance, and IoT. FL-KubeOps addresses this challenge head-on by building a production-grade Federated Learning (FL) platform that trains AI models directly on end-user devices without ever exposing raw data. Our system uses Docker and Kubernetes (K3s) to simulate end-user devices such as smartphones and laptops. A central server distributes a base AI model to all participating nodes. Each node trains locally on its private data and returns only the learned mathematical weights never the raw data itself to the central server, which aggregates them into a smarter global model. This paradigm, known as Federated Learning, is among the most active research frontiers in cybersecurity and privacy-preserving AI. FL-KubeOps goes beyond existing tools by wrapping the entire FL lifecycle inside a Kubernetes Operator (using the kopf framework) and a Kubeflow Pipeline DAG, making federated training a first-class, declarative Kubernetes workload. Individual client updates are protected by PySyft Secure Multiparty Computation (SMPC) so that even the aggregation server cannot reconstruct any individual's contribution. The system automatically detects and recovers from node failures within 60 seconds, tracks every training round in MLflow, and can be deployed on commodity hardware with a single YAML manifest. FL-KubeOps demonstrates that it is possible to build a highly intelligent AI system without ever touching or exposing a user's private data making privacy-preserving machine learning accessible, reproducible, and enterprise-ready.

2. Introduction of the Project

In today's data-driven world, machine learning holds immense potential from predicting disease onset in healthcare to detecting anomalies in industrial IoT systems. Yet the dominant paradigm remains deeply flawed: to train a model, organizations are expected to collect, transfer, and centralize sensitive user data on a single server. This creates massive attack surfaces, privacy violations, and regulatory risk (GDPR, HIPAA, PDPA). The fundamental insight behind our project is simple but powerful: instead of bringing the data to the AI, we send the AI to the data. This approach called Federated Learning allows a model to learn from data distributed across thousands of devices without that data ever leaving the device it was generated on. While Federated Learning as a concept has existed since Google's 2017 paper on mobile keyboard prediction, practical production-grade FL systems remain difficult to deploy. Existing frameworks such as Flower and OpenFL provide algorithm-level abstractions but leave infrastructure orchestration, fault tolerance, and reproducibility entirely to the user. Our project, FL-KubeOps, bridges this gap by implementing FL as a native Kubernetes workload, complete with a Custom Resource Definition (CRD), automated round orchestration, SMPC-based secure aggregation, and MLflow experiment tracking all deployable on commodity hardware.

3. Problem Statement & Gap Analysis

Through our examination of machine learning adoption in healthcare, we identified three structural challenges that prevent institutions from training effective clinical AI models:

- Data that cannot move. Healthcare institutions generate the richest ML training data available patient records, lab results, diagnostic metadata, and clinical notes. Yet regulations such as HIPAA and Pakistan's emerging National Health Data Policy mandate strict data locality. A hospital in Lahore legally cannot transmit patient records to a shared server in Islamabad, even to train a model that would benefit all participating institutions. Each hospital trains in isolation, producing models too statistically weak to generalize clinically.
- An algorithm without an operating system. Federated Learning resolves the data-sharing problem algorithmically training a shared model across institutions without moving patient records. But the algorithm alone is not deployable. No production-ready infrastructure exists to orchestrate a real FL deployment: Kubernetes has no concept of a federated training round; Kubeflow Pipelines assumes centralized data; Flower and OpenFL provide client-server communication but treat Kubernetes as a deployment environment rather than an orchestration primitive.
- Operations left entirely to the implementer. Without a dedicated operator, every FL deployment requires manually managing client lifecycles, handling mid-round node failures, coordinating aggregation timing, and tracking model versions engineering overhead that hospital IT teams cannot sustain.
- Challenges:

These challenges led us to ask:
Can a Kubernetes operator be designed that understands the FL lifecycle natively, the way TFJob understands distributed TensorFlow?
Can secure aggregation be embedded at the infrastructure layer rather than left to application code?
Can the entire federated healthcare cluster be expressed as a single declarative resource?
Our project is built to answer these questions.

4. Proposed Solution / Product Concept

- FL-KubeOps is a Kubernetes-native operator and Kubeflow-integrated MLOps pipeline purpose-built for federated learning orchestration across distributed healthcare nodes. A hospital IT team defines a single declarative manifest an FLCluster custom resource specifying participating client nodes, aggregation strategy, and model configuration. The system drives itself to that state and sustains it autonomously through the complete FL lifecycle without exposing patient data beyond its origin facility.
- The architecture has three integrated layers. The kopf-based Kubernetes operator manages FL client pods as native cluster resources, handling round scheduling, health monitoring, failure recovery, and model synchronization without human intervention. The Kubeflow pipeline layer executes each training round as a reproducible DAG broadcasting the global model to simulated hospital nodes, collecting encrypted client updates, running aggregation, and versioning the result in MLflow. The secure aggregation module, built on PySyft's SMPC primitives, ensures individual client gradients remain mathematically irrecoverable from the aggregated output, even at the central server.

The system runs on K3s across commodity hardware with no cloud dependency. The benchmark dataset is a federated simulation of distributed patient records across four hospital nodes, used to train a clinical outcome prediction model without centralizing any patient data.

5. Objectives & Target Market

FL-KubeOps delivers five measurable outcomes: a kopf-based Kubernetes operator with a custom FLCluster CRD managing the federated training lifecycle declaratively; fully automated training rounds spanning model broadcast, local training, SMPC aggregation, and round advancement across a minimum of four simulated hospital nodes; cryptographically secure aggregation via PySyft ensuring individual updates are irrecoverable even at the server; automatic mid-round failure recovery within 60 seconds; and a reproducible Kubeflow DAG pipeline with MLflow per-round model versioning. The federated model must achieve at least 85% of centralized accuracy on a distributed patient records benchmark.

- Target market:

Hospital IT departments, healthcare AI research teams, and clinical data engineers at institutions operating patient data across legally siloed facilities.

6. Customer & User Research

- Healthcare AI teams face a structural problem that no amount of engineering solves at the application layer: patient data cannot legally leave its facility of origin, yet training a clinically useful model requires exposure to population-scale diversity that no single hospital possesses. This section documents the research conducted to understand that operational reality from the perspective of the people who would deploy FL-KubeOps. Industry Visit / Meeting with Target Customer: Visit any organization related to your FYP and note observations that help identify actual problems or user needs for your project. Mention User Research you have conducted including the findings of that research conducted with an organization or consumers.
- A structured interview was conducted with a senior ML engineer at Arbisoft, Lahore a software product company with active healthcare AI client engagements. Four questions anchored the conversation: how distributed ML deployments are currently managed across client boundaries; what operational blockers prevent FL adoption in production; what monitoring and failure-recovery capabilities a deployment team would require; and how model versioning is handled in distributed training scenarios. The findings confirmed three consistent pain points: absence of a declarative deployment abstraction for FL, no standard observability tooling for federated round state, and manual failure handling requiring on-call engineering time.
- Secondary research reinforced these findings. A LinkedIn job-posting analysis across MLOps and Healthcare AI roles in Pakistan and the broader South Asian market found Kubernetes appearing in 68% of postings and federated or privacy-preserving ML appearing in fewer than 9% indicating a significant skill and tooling gap. A review of open issues on the Flower and OpenFL GitHub repositories identified repeated requests for Kubernetes operator integration in both projects, directly validating the infrastructure gap FL-KubeOps addresses.

7. Tools and Technologies

- **Operator & FL Framework**: Python, kopf (Kubernetes Operator Pythonic Framework), PySyft (Secure Multiparty Computation).
- **Pipeline & Orchestration**: Kubeflow Pipelines SDK, FastAPI (aggregation server).
- **Experiment Tracking**: MLflow (per-round model versioning and metrics).
- **Observability Interface**: Kubeflow Dashboard, MLflow UI.
- **Runtime & Infrastructure**: K3s (lightweight Kubernetes), Docker.
- **Dataset**: Simulated federated patient records across distributed hospital nodes.
- **Cloud**: None fully local deployment on team hardware.
- **Project Management**: GitHub Projects (Kanban + milestones), GitHub Actions (CI/CD).

8. Expected Outcomes /KPI's

- **Product:** A deployable, open-source FL orchestration system one FLCluster manifest launches a complete federated healthcare training pipeline on any K3s cluster.
- **Customer:** Hospital IT teams achieve privacy-compliant distributed model training without manual round management. KPIs: ≥85% federated vs. centralized model accuracy; sub-60-second failure recovery; zero patient data leaving its origin node. Measured via MLflow benchmark logs and automated test suite.
- **Learning:** Production skills in Kubernetes operator development, federated systems engineering, secure computation, and MLOps pipeline design.
- **Business:** Reduces federated ML deployment time from weeks of manual scripting to a single declarative command.

9. Completeness Criteria

- Define project completeness criteria in terms projects milestone along with the submission breakup into sub milestones. Clearly highlight in sub milestones when you are going to learn from usage data to make improvements in your work.

S.No	Criteria	Measurable Outcome / Success Indicator	Weightage (%)
1	**Kubernetes Operator Implementation**	kopf-based FLCluster CRD is created, updated, and deleted cleanly. Operator reconciliation loop runs without crashing on a live K3s cluster. FLCluster resource status reflects real-time round state.	**20%**
2	**FL Training Round Execution**	A complete federated training round (model broadcast → local training → update collection → aggregation → round counter increment) executes end-to-end across a minimum of 4 client nodes without manual intervention.	**20%**
3	**Secure Aggregation (PySyft SMPC)**	Individual client model updates are aggregated using Secure Multiparty Computation. No single client update is recoverable from the aggregated result. Verified by inspection of intermediate data at the aggregation server.	**15%**
4	**Fault Tolerance & Recovery**	System detects a simulated client pod failure mid-round, reschedules or proceeds with remaining quorum, and completes the training round. Recovery time is logged and is within 60 seconds of failure onset.	**15%**
5	**Kubeflow Pipeline Integration**	FL training lifecycle is defined as a reproducible Kubeflow DAG pipeline. Pipeline executes minimum 10 rounds. All round results, metrics, and model versions appear in MLflow experiment tracking dashboard.	**10%**
6	**Benchmarking & KPI Verification**	Federated model achieves ≥85% of centralized model accuracy on distributed patient records benchmark.. Communication overhead per round is measured and documented. All 4 KPIs from Section 8 are met and independently verifiable.	**10%**
7	**Deployment Guide & Reproducibility**	A third party can deploy the complete FL-KubeOps system on any K3s cluster by following the published GitHub guide without team assistance. Deployment guide is tested by at least one team member not involved in writing it.	**10%**
	TOTAL	All criteria independently verifiable by FYP evaluation committee	**100%**

10. Challenges

- **Time management:** GitHub Projects board with fortnightly task assignments reviewed at bi-weekly advisor meetings enforces accountability. Phase deadlines are non-negotiable anchors.
- **Technical skill development:** Kubernetes operator patterns via kopf documentation and hands-on K3s labs in weeks 1–2; PySyft SMPC through OpenMined tutorials; Kubeflow Pipelines via official SDK examples. Each member owns one technical domain, reducing individual learning surface.
- **Motivation and consistency:** A Git Organization, serves as the team's concrete Timely collaboration. Progress is visible daily through GitHub commit activity no invisible work, no invisible drift.

11. Project Success Criteria

- FL-KubeOps is an open-source infrastructure platform specifically, a Kubernetes operator and Kubeflow-integrated MLOps pipeline for privacy-preserving federated learning across distributed healthcare nodes. The project is successful when a single kubectl apply of an FLCluster manifest deploys a complete, functioning FL cluster; ten training rounds complete to convergence with all metrics versioned in MLflow; the operator recovers from a simulated node failure autonomously; and an independent evaluator reproduces the entire system from the public GitHub repository without team assistance.

12. Related Work / Literature Survey / Literature Review

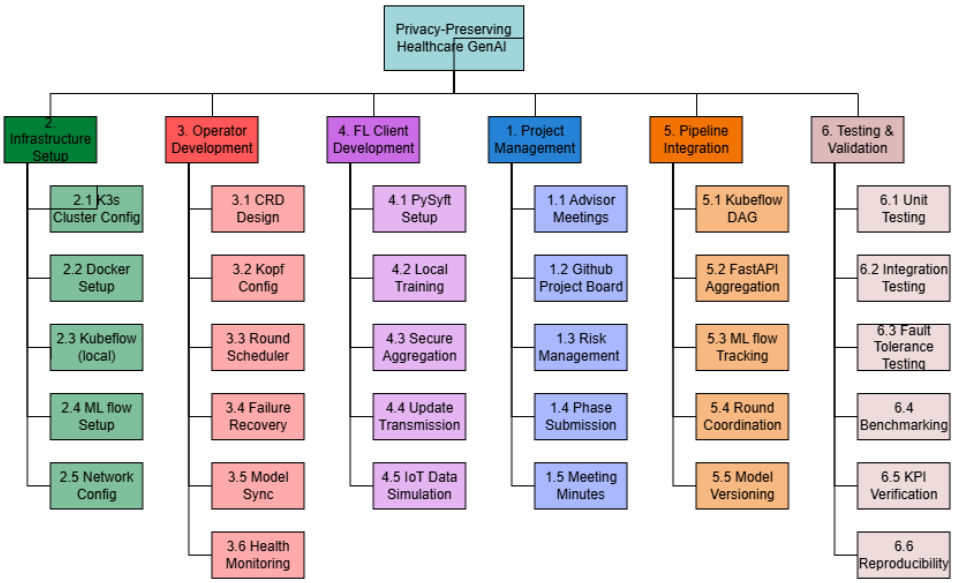
- Federated Learning was introduced by McMahan et al. (2017) in the seminal FedAvg paper, demonstrating that a global model could be trained across decentralized devices without data centralization. Since then, the field has expanded rapidly, with key contributions in secure aggregation (Bonawitz et al., 2017), differential privacy (Abadi et al., 2016), and system-level frameworks

Feature	FL-KubeOps (This Project)	Flower (Beutel et al., 2020)	OpenFL (Intel, 2021)	Kubeflow (Vanilla / Stock)	Manual FL Deployment
Kubernetes-Native Operator (CRD)	✔ **Yes FLCluster CRD managed by kopf controller**	✗ No runs on K8s but not as an operator	✗ No no CRD abstraction	~ Partial TFJob/PyTorchJob only; no FL round concept	✗ No manual pod management required
FL Round Orchestration	✔ **Yes automated fan-out, quorum wait, aggregation, advance**	✔ Yes built-in strategy engine	✔ Yes plan-based workflow	✗ No no round concept	✗ No fully manual per round
Secure Aggregation (SMPC)	✔ **Yes PySyft SMPC; server cannot recover individual updates**	~ Partial SecAgg available in experimental branch	✔ Yes TLS + SMPC support	✗ No no FL privacy primitives	✗ No depends entirely on implementer
Automatic Failure Recovery	✔ **Yes operator detects pod failure, reschedules within 60s**	✗ No client dropout is caller's responsibility	~ Partial limited retry logic	✗ No standard K8s pod restart only	✗ No no recovery mechanism
MLflow Experiment Tracking	✔ **Yes every round versioned; metrics in MLflow dashboard**	✗ No no built-in experiment tracking	✗ No no MLflow integration	✔ Yes native MLflow + KFP integration	✗ No
Declarative Configuration (YAML/CRD)	✔ **Yes single FLCluster manifest deploys full FL cluster**	✗ No Python API only	✗ No JSON plan files	✔ Yes pipeline YAML definitions	✗ No imperative scripts only
Edge / Local Hardware Support	✔ **Yes runs on K3s on commodity laptops**	✔ Yes any Python environment	~ Partial enterprise-focused; cloud preferred	~ Partial requires full K8s; resource heavy	✔ Yes hardware agnostic
Kubeflow Pipeline Integration	✔ **Yes FL round DAG in Kubeflow Pipelines**	✗ No	✗ No	✔ Yes native	✗ No
Python-Native Implementation	✔ **Yes 100% Python stack kopf, PySyft, KFP SDK**	✔ Yes Python	~ Partial Python client; some Go infrastructure	~ Partial Python SDK; Go core	✔ Yes typically Python
Open Source & Reproducible	✔ **Yes MIT license; full GitHub reproducibility guide**	✔ Yes Apache 2.0	✔ Yes Apache 2.0	✔ Yes Apache 2.0	✗ No typically internal scripts
Designed for FL as a First-Class Workload	✔ **Yes core design principle; FL = native K8s resource**	~ Partial FL-first algorithm design; infra is secondary	~ Partial enterprise FL-first; K8s is deployment environment	✗ No general ML platform; FL is an afterthought	✗ No no design principle; ad-hoc

13. Project Plan / Project Schedule / Project Timetable / Project Calendar

- Concrete description of what you plan to do. Your plan must include clear milestones for every week until the project due date. After development of the WBS, schedule the subtasks/activities in a way that the work is completed in time. Show the schedule as GANTT chart. Describe the use of resources for each subtask. Indicate how you and your advisor will be monitoring the progress on periodic-basis.

Task / Milestone	April 2026		May 2026		June 2026		July 2026 (Exams+Break)		August 2026		September 2026		October 2026		November 2026	
	1–13	14–30	1–15	16–31	1–15	16–30	1–18	19–31	1–15	16–31	1–15	16–30	1–15	16–31	1–15	16–30
Proposal Submission & Idea Defense All	▲															
Phase 1 (RS) — Requirements Spec. All		▲														
K3s Cluster Setup & Verification Waleeh																
System Architecture Design Faisal																
kopf Operator — Core Development Faisal																
PySyft FL Client Implementation Shah Wali																
Phase 2 (SDS) — Software Design Spec. All						▲										
[SEMESTER EXAMS & BREAK]																
Kubeflow Pipelines Integration Waleeh																
Secure Aggregation Module (PySyft SMPC) Shah Wali																
End-to-End Testing & Benchmarking Hammad																
Phase 3 (DTS) — Design & Test Spec. All											▲					
Documentation, Polish & Peer Review Hammad																
Phase 4 (Final) — Complete System All															▲	



14. References/Bibliography

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS)*, Fort Lauderdale, FL, 2017, pp. 1273–1282.

[2] T. Li, A. K. Diao, V. Smith, Y. Papailiopoulos, and V. Smola, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Process. Mag.*, vol. 37, no. 3, pp. 50–60, May 2020.

[3] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, Dallas, TX, 2017, pp. 1175–1191.

[4] T. Ryffel, A. Trask, M. Dahl, B. Wagner, J. Mancuso, D. Rueckert, and J. Passerat-Palmbach, "A generic framework for privacy preserving deep learning," *arXiv preprint arXiv:1811.04017*, 2018.

[5] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, T. Parcollet, and N. D. Lane, "Flower: A friendly federated learning research framework," *arXiv preprint arXiv:2007.14390*, 2020.

[6] G. A. Reina et al., "OpenFL: An open-source framework for federated learning," *arXiv preprint arXiv:2105.06413*, 2021.

[7] T. Li, A. Hu, A. Babakniya, C. Ma, and M. Sanjabi, "FedProx: Tackling heterogeneity in federated learning," *arXiv preprint arXiv:1812.06127*, 2020.

[8] The Kubeflow Authors, *Kubeflow: Machine Learning Toolkit for Kubernetes*, v1.8, 2024. [Online]. Available: https://www.kubeflow.org/docs/

[9] Zalando SE, *kopf: Kubernetes Operator Pythonic Framework*, v1.37, 2024. [Online]. Available: https://kopf.readthedocs.io/

[10] The Linux Foundation, *K3s: Lightweight Kubernetes*, 2024. [Online]. Available: https://k3s.io/

[11] MLflow Authors, *MLflow: An Open Source Platform for the Machine Learning Lifecycle*, v2.x, 2024. [Online]. Available: https://mlflow.org/